

# Defesa do Projeto WiSafe / Wifi Protect

## 1. Abertura

O WiSafe, também chamado de Wifi Protect, é um aplicativo mobile criado para ajudar o usuário a identificar redes Wi-Fi próximas e entender se elas são seguras antes de tentar se conectar.

O projeto atende aos requisitos obrigatórios do trabalho e vai além deles, porque usa um recurso nativo real do Android para fazer a varredura das redes Wi-Fi disponíveis no ambiente.

Tempo sugerido: 1 minuto.

## 2. Sequencia da apresentacao

1. Apresentar o objetivo do app.
2. Mostrar os requisitos obrigatórios atendidos.
3. Mostrar as telas e a navegação.
4. Explicar login, cadastro e controle de acesso.
5. Explicar banco de dados e armazenamento local.
6. Explicar a parte de redes Wi-Fi e recurso nativo Android.
7. Mostrar a classificação de risco e os avisos de segurança.
8. Finalizar com os diferenciais do projeto.

Tempo total sugerido: até 15 minutos.

## 3. Telas e navegacao

Arquivos principais:

- src/screens/HomeScreen.tsx
- src/screens/LoginScreen.tsx
- app/registro.tsx
- src/screens/NetworkScreen.tsx
- src/screens/NetworkDetailsScreen.tsx
- src/screens/FavoritesScreen.tsx
- src/screens/AccountScreen.tsx
- src/screens/LogoutConfirmScreen.tsx

Funções principais para apresentar:

```
export default function HomeScreen()
```

Essa função representa a tela inicial do app. Ela mostra o carrossel com imagens, o botão para começar e o botão para criar conta.

```
export default function LoginScreen()
```

Essa função representa a tela de login. Nela o usuário informa e-mail e senha para

acessar o aplicativo.

```
export default function NetworksScreen()
```

Essa e a tela principal depois do login. Ela mostra as redes Wi-Fi disponiveis, o nome do usuario e o botao de escanear.

```
export default function NetworkDetailsScreen()
```

Essa tela mostra os detalhes da rede selecionada, como nome, seguranca, sinal, frequencia e nivel de risco.

Codigo de navegacao:

```
router.push("/login")  
router.push("/registro")  
router.replace("/home")  
router.push("/network-details")
```

Explicacao para falar:

O app usa Expo Router para navegar entre as telas. O router.push abre uma nova tela, enquanto o router.replace troca a tela atual, evitando que o usuario volte para uma tela indevida, como aconteceu no caso do logout.

## 4. Controle de acesso

Arquivo principal:

- src/screens/LoginScreen.tsx

Funcao principal:

```
async function handleEntrar()
```

Essa funcao valida os campos de login, chama o Firebase Authentication e, se o login estiver correto, envia o usuario para a tela de redes.

Codigo importante:

```
await signInWithEmailAndPassword(auth, email.trim(), senha)
```

Explicacao para falar:

O controle de acesso foi feito com Firebase Authentication. Isso significa que o app nao libera a tela principal sem antes validar o e-mail e a senha do usuario.

Tambem existe a opcao "Lembrar de mim", que salva o login no dispositivo.

## 5. Cadastro e coleta de dados

Arquivo principal:

- app/registro.tsx

Funcao principal:

```
async function handleRegistrar()
```

Essa funcao coleta nome, e-mail e senha. Depois cria o usuario no Firebase e salva os dados basicos no Firestore.

Codigo importante:

```
await createUserWithEmailAndPassword(auth, email.trim(), senha)

await setDoc(doc(db, "usuarios", user.uid), {
  nome,
  email,
})
```

Explicacao para falar:

A coleta de dados acontece no cadastro. O app coleta nome, e-mail e senha. A senha e usada pelo Firebase Authentication, e os dados como nome e e-mail sao armazenados no Firestore.

## 6. Banco de dados

Arquivo principal:

- src/services/firebase.ts

Codigo importante:

```
export const auth = getAuth(app);
export const db = getFirestore(app);
```

Explicacao:

O auth e usado para login, cadastro, recuperacao de senha e logout. O db representa o Firestore, usado para guardar dados do usuario.

Na defesa, fale:

O projeto usa Firebase em duas partes. A primeira e o Authentication, que cuida do acesso do usuario. A segunda e o Firestore, que armazena dados da conta, como nome e e-mail.

## 7. Armazenamento no dispositivo

Arquivos principais:

- src/storage/userStorage.ts
- src/storage/favoritesStorage.ts
- src/storage/localStorage.ts

Funcoes do usuario:

```
saveStoredUser(user)
getStoredUser()
clearStoredUser()
```

Explicacao:

saveStoredUser salva o usuario no dispositivo. getStoredUser busca esse usuario salvo

quando o app abre. `clearStoredUser` apaga esses dados no logout.

Funcoes dos favoritos:

```
getFavoriteNetworks()  
saveFavoriteNetwork(network)  
removeFavoriteNetwork(bssid)  
toggleFavoriteNetwork(network)
```

Explicacao:

Essas funcoes controlam as redes favoritas no armazenamento local. Assim, quando o usuario marca uma rede como confiavel, essa informacao continua salva no aparelho.

Na defesa, fale:

O app usa armazenamento local para melhorar a experiencia. Ele salva o login quando o usuario escolhe lembrar e tambem guarda redes favoritas no proprio dispositivo.

## 8. Redes Wi-Fi

Arquivo principal:

- `src/services/wifiService.ts`

Funcoes principais:

```
scanWifiNetworks(forceRefresh = false)
```

Essa funcao busca as redes Wi-Fi proximas. Quando `forceRefresh` e verdadeiro, ela pede uma nova varredura.

```
findWifiNetwork(bssid)
```

Essa funcao procura uma rede especifica pelo BSSID, que funciona como um identificador da rede.

```
connectToWifiNetwork(network, password)
```

Essa funcao envia para o Android o pedido de conexao com a rede escolhida.

Codigo importante:

```
const nativeScanner = NativeModules.WifiScanner;  
const networks = await nativeScanner.scanWifiNetworks(forceRefresh);
```

Explicacao para falar:

A interface do app foi feita em React Native, mas a busca real das redes depende de um modulo nativo Android. O React Native chama esse modulo usando `NativeModules`.

## 9. Recurso nativo Android

Arquivo principal:

- `android/app/src/main/java/com/anonymous/matheus123/WifiScannerModule.kt`

Funcoes principais:

```
fun scanWifiNetworks(forceRefresh: Boolean, promise: Promise)
```

Essa funcao usa o WifiManager do Android para iniciar a varredura de redes Wi-Fi proximas.

```
fun connectToNetwork(ssid: String, securityType: String, password: String, promise: Promise)
```

Essa funcao solicita ao Android a conexao com uma rede Wi-Fi.

Codigo importante:

```
val wifiManager = applicationContext.getSystemService(Context.WIFI_SERVICE) as WifiManager
wifiManager.startScan()
```

Explicacao para falar:

Esse e o recurso nativo mais importante do projeto. O app nao mostra apenas redes fixas na tela. Ele usa o Android para procurar redes reais proximas do celular.

## 10. Regras de seguranca

Arquivo principal:

- src/services/wifiRules.ts

Funcoes principais:

```
getSecurityType(capabilities)
```

Identifica se a rede e aberta, WEP, WPA, WPA2, WPA3 ou desconhecida.

```
classifyRisk(securityType)
```

Classifica o risco da rede:

- OPEN: alto risco
- WEP: medio risco
- WPA: medio risco
- WPA2: seguro
- WPA3: seguro

```
getSignalLabel(level)
```

Transforma o nivel do sinal em uma informacao mais facil para o usuario: forte, medio ou fraco.

```
getSecurityWarning(riskLevel)
```

Gera o texto de alerta mostrado para o usuario.

Explicacao para falar:

Depois que o Android retorna as redes, o app interpreta o tipo de seguranca e transforma isso em uma classificacao simples: seguro, medio risco ou alto risco.

## 11. Aviso antes de conectar

Arquivo principal:

- src/screens/NetworkDetailsScreen.tsx

Funcoes principais:

`handleConnectPress()`

Verifica se a rede e segura. Se for de risco medio ou alto, abre o aviso antes de continuar.

`continueToConnection()`

Continua o processo depois que o usuario confirma que entendeu os riscos.

`requestConnection(password)`

Chama a funcao de conexao e envia o pedido para o Android.

Explicacao para falar:

O app nao conecta automaticamente em redes de risco. Primeiro ele mostra um aviso em duas etapas. Isso ajuda o usuario a entender que redes abertas ou antigas podem expor dados pessoais.

## 12. Componentes visuais

O projeto usa:

- imagens no carrossel;
- imagem do icone wifi-protect.jpeg no cabecalho;
- botoes;
- campos de texto;
- checkbox "Lembrar de mim";
- cards de redes;
- listagem de redes;
- modal de aviso;
- snackbar para mensagens;
- icones de cadeado, Wi-Fi, favorito e conta.

Explicacao para falar:

A interface foi pensada para ser simples. O usuario consegue criar conta, entrar, visualizar redes, favoritar redes e receber avisos sem precisar entender termos tecnicos.

## 13. Requisitos atendidos

- Quatro telas: atendido, o app tem mais de quatro.
- Controle de acesso: login, cadastro e logout.
- Coleta de dados do usuario: nome, e-mail e senha.
- Armazenamento no dispositivo: login salvo e redes favoritas.
- Recurso nativo: varredura real de redes Wi-Fi pelo Android.
- Componentes visuais: imagens, botoes, campos, checkbox, cards, modais e listagens.

## 14. Diferenciais

O projeto vai alem do minimo porque:

- usa modulo nativo Android em Kotlin;
- escaneia redes Wi-Fi reais;
- classifica redes por nivel de risco;
- mostra aviso antes de conexao insegura;
- salva login e favoritos no dispositivo;
- usa Firebase para autenticar e armazenar dados do usuario;
- gera APK usando EAS.

## **15. Fechamento**

Fala sugerida:

O projeto atende aos requisitos obrigatorios e vai alem deles, porque transforma a proposta em um aplicativo funcional. Ele possui autenticacao, coleta de dados, armazenamento local, banco de dados externo, telas navegaveis e um recurso nativo Android para buscar redes Wi-Fi reais e avaliar o risco de seguranca.